DataGuard AI: Visual Telemetry Fingerprinting and Risk-Governed Agentic Data-Quality Incident Response

Tishya Jha
Department of Computer Science and Design Engineering
Rajiv Gandhi Institute of Petroleum Technology
23cd3049@rgipt.ac.in

Aarna Kumar
Department of Computer Science and Design Engineering
Rajiv Gandhi Institute of Petroleum Technology
23cd3001@rgipt.ac.in

***Abstract*—Data pipelines may finish successfully while emitting incomplete, duplicated, schema-incompatible, or distribution-shifted outputs. DataGuard AI is an implemented local Python modular monolith connecting structured quality checks, Isolation Forest, evidence-grounded root-cause analysis (RCA), local runbook retrieval, guarded simulated actions, verification, persistence, FastAPI, and an API-backed Streamlit dashboard. Its digital image processing module encodes a Data Quality Fingerprint (DQF), with a separate validity mask for missing telemetry, and maps OpenCV regions back to run and metric identifiers. On the reproducible 360-run synthetic benchmark, the 72-run test split produced F1=0.800 for the severity baseline and F1=0.732 (AP=0.591) for structured Isolation Forest. On a matched 43-run subset, adding DQF-region features raised F1 from 0.789 to 0.862 while AP fell from 0.764 to 0.673. Across three held-out fingerprints (1,024 valid pixels), global Otsu segmentation reached F1=0.779 and IoU=0.638; the adaptive/morphological alternative scored 0.324 and 0.194. Fixed local evaluations measured runbook hit@3 at 6/7 and RCA top-1/top-3 accuracy at 4/5 and 5/5. These synthetic results demonstrate a working prototype and also expose its limitations.**

***Index Terms*—Data quality, anomaly detection, digital image processing, root-cause analysis, retrieval-augmented generation, human-in-the-loop systems.**

I. Introduction

Data reliability cannot be inferred from process completion alone: a successful pipeline execution may still introduce null surges, duplicate keys, row-count drops, schema drift, or population shifts. Such failures propagate through dependent datasets and can remain latent until downstream analytics are affected. Rule-based checks are interpretable but often leave operators to correlate telemetry, schema history, lineage, prior incidents, and remediation outcomes manually. A reliable incident-response system must preserve evidence, distinguish observations from causal hypotheses, control state-changing actions, and verify the post-action state rather than merely generate an explanation.

DataGuard AI implements this closed loop in one Python modular monolith: normalised telemetry feeds structured quality checks and anomaly detection; DQF regions add visual evidence; RCA combines findings with schema, lineage, and history; runbook retrieval informs a typed plan; and a server-side policy guard gates action before verification and persistence.

The contributions are an auditable pixel-to-telemetry encoding with explicit missingness, a comparative classical-DIP pipeline, a single orchestrator with typed tools and mandatory approval for high-risk actions, and an evaluation spanning detection, segmentation, retrieval, tool safety, and latency. The system is evaluated on a seeded synthetic benchmark; its results and limitations are reported in Section V.

II. Related Work and State of the Art

Automated data-quality systems combine declarative constraints, historical metrics, and anomaly detection [1]; Isolation Forest provides an efficient unsupervised score through random partitioning [3]. Recent image-encoded time-series work uses learned representations to localise sensor anomalies [2]. DataGuard AI instead uses a calibrated DQF with classical OpenCV operators and retains structured findings as the authoritative quality evidence.

Image encodings have been used for time-series inspection and classification [6]; Otsu thresholding supplies a global baseline [4], and morphology analyses foreground shape [5]. DQF interpretation requires stable intensity calibration, channel order, missingness, and pixel-to-telemetry mapping. Accordingly, the evaluation scores saved masks against injected-defect labels rather than treating a dashboard heatmap as evidence by itself.

Retrieval-augmented generation grounds generated responses in retrieved passages [7], while reasoning-and-acting research motivates iterative evidence collection and action selection [8]. For guarded remediation, however, a language model must not execute arbitrary code or override policy. DataGuard AI narrows these ideas into a single state machine, source-cited runbook retrieval, explicit tool contracts, server-side approval checks, and post-action verification. The research gap addressed is the integration of structured quality checks, visual evidence, evidence-grounded RCA, runbook references, risk-governed action, and persisted outcome verification in one reproducible local prototype.

III. Proposed Methodology and Architecture

A. Modular Closed-Loop Design

The system follows four dependency layers: domain contracts, application use cases, infrastructure adapters, and presentation. Ingestion and normalisation have one owner; `quality.py` evaluates rules once; a canonical feature builder supplies a stable named vector; `visual_inspection.py` owns DQF encoding and DIP; detection and incident services deduplicate findings; RCA ranks hypotheses; RAG owns chunking and retrieval; and a single `AgentOrchestrator` owns state transitions. FastAPI exposes application use cases, whereas Streamlit acts only as an API client. SQLite stores relational entities and audit events; image arrays and rendered artifacts are stored as files with stable relative references.

B. Canonical Quality Evidence and Detection

Ingestion validates timestamps, row counts, duration, schema version, source/target identifiers, and field-level metrics, then creates one canonical run snapshot. The quality engine performs six classes of checks: null/missing rates, duplicate-key rates, input/output row-count deltas, schema additions/removals/renames/type changes, invalid type/range/value checks, and numerical distribution deviation. Each finding records its rule ID, severity, observed and expected values, evidence, and timestamp. The feature builder creates a stable named vector from run telemetry and findings. A deterministic threshold baseline is compared with an Isolation Forest model; its anomaly score is not interpreted as a calibrated probability. Training, validation, and test partitions are grouped chronologically or by incident, and the decision threshold is selected on validation data and frozen before testing.

C. Data Quality Fingerprint and DIP

For a fixed history window, DQF rows correspond to ordered runs and columns to a versioned list of (field_id, metric_id) channels. Let $x_{r,c}$ be a metric, m_c its healthy reference (e.g., the training median), and s_c a robust channel scale (e.g., the median absolute deviation) estimated from training data. Its calibrated deviation and 8-bit intensity are

$$d_{r,c} = \min\left(1, \frac{|x_{r,c} - m_c|}{\kappa s_c + \epsilon}\right), \quad I_{r,c} = \mathrm{round}(255 d_{r,c}), \quad (1)$$

where κ and ϵ are configured. Channel order, baseline version, image dimensions, run IDs, and channel IDs are persisted. A validity mask $M \in \{0,1\}^{W \times C}$ distinguishes missing observations from low risk.

The implemented DIP-B0 applies Otsu global thresholding and connected-component analysis. DIP-P1 uses adaptive thresholding, a 3×3 elliptical morphological closing, and connected-component analysis; median filtering is disabled because the generator adds no impulse noise. Each region records area, bounding box, centroid, temporal/channel spans, peak intensity, density, and mapped run/metric IDs. The evaluated DQF is 30×12 with adaptive block size 11 and offset -2. Encoder metadata, configuration, intermediate images, masks, overlays, and region CSVs are persisted with each artifact.

DIP procedure: (1) encode the chronological window and validity mask; (2) produce DIP-B0 and DIP-P1 masks; (3) label connected regions and map coordinates to source run/channel IDs; (4) save the raw image, masks, overlay, region table, configuration, and processing duration; (5) compare both methods against injected-defect masks using precision, recall, F1, and IoU or Dice. Three configured DQF-region features (maximum persistence, region density, and affected-channel count) were included in a matched held-out detector ablation. A segmented region is evidence of a pattern, not proof of causation.

D. Investigation, Retrieval, and Guarded Action

RCA ranks hypotheses from quality findings, schema diffs, dependency lineage, recent runs, historical incidents, action outcomes, and optional DQF regions. A row-count drop paired with an upstream failure supports an upstream-dependency hypothesis; a type mismatch aligned with a schema change supports schema drift. Conflicting or sparse evidence yields `INCONCLUSIVE` rather than a fabricated cause.

Seven local Markdown runbooks are chunked with stable source IDs and retrieved by deterministic TF–IDF cosine similarity. Retrieved chunks retain document ID, section ID, score, and source text. A single `AgentOrchestrator` controls transitions `NEW`→`INVESTIGATING`→`PROPOSED`, followed by approval or execution readiness, `EXECUTING`, and `VERIFYING`, ending in `RESOLVED`, `ESCALATED`, or `FAILED`. Each typed plan includes tool, arguments, preconditions, evidence/runbook references, risk, and expected outcome.

The allowlist includes read-only inspection and retrieval, reversible quarantine, simulated rerun, sandboxed simulated repair, and escalation. Quarantine is idempotent; reruns do not invoke a real scheduler; repairs preserve the original artifact. Ambiguous or high-risk actions require an explicit approval record, checked server-side at execution time. Each transition and tool result is audited with correlation and incident IDs. Verification reruns the relevant checks; failed verification leaves the incident unresolved or escalates it.

1: Validate telemetry; persist the canonical run and compute quality findings/features
2: Encode DQF; run DIP-B0/DIP-P1; persist masks and mapped regions
3: Detect and deduplicate the incident; rank evidence-backed RCA hypotheses and retrieve cited runbook sections
4: Build a typed plan; enforce tool allowlist, preconditions, risk policy, and required approval
5: Execute the local/simulated action; re-check quality and persist verification, audit trace, and outcome

IV. Implementation and Experimental Setup

DataGuard AI is implemented and runnable as a Python modular monolith. Modules cover strict JSONL ingestion and synthetic data generation, schema/lineage, one structured quality engine, canonical features, DQF encoding and OpenCV processing, Isolation Forest and severity-baseline

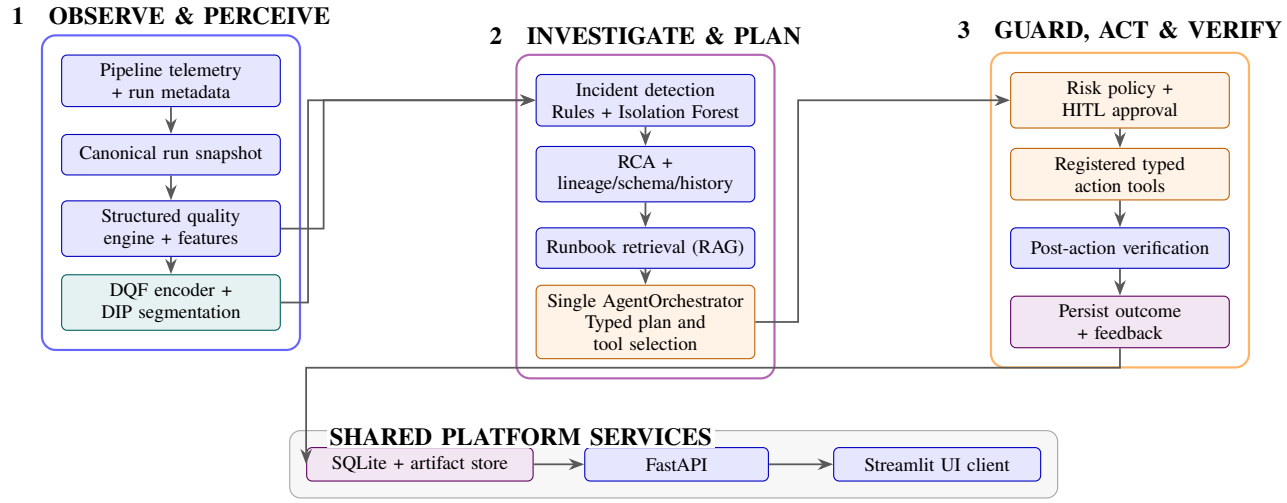


Fig. 1. DataGuard AI architecture. The DQF/DIP channel contributes mapped evidence to detection and investigation; structured quality rules remain authoritative. All state-changing actions pass through policy and typed tools before verification and persistence.

scoring, incident deduplication, evidence-ranked RCA, TF–IDF runbook retrieval, a single orchestration state machine, typed action tools and server-side approval, post-action checks, SQLite JSON-record persistence, FastAPI routes, Streamlit screens, and saved evaluation artifacts. Runtime dependencies are NumPy, OpenCV, scikit-learn, FastAPI, Pydantic Settings, Streamlit, Uvicorn, and Joblib; Python's standard-library SQLite driver handles persistence. YAML and environment configuration own thresholds, model parameters, DIP settings, approval policy, database paths, and artifact paths.

Eight registered tool operations provide inspection, retrieval, quarantine/release, simulated rerun, schema-alignment simulation, escalation, and approval recording. FastAPI exposes health, demo generation, run inspection/analysis, incident investigation/planning, approval, action execution, timelines, DQF artifact allowlists, and evaluation summaries under `/api/v1`. SQLite persists runs, findings, incidents, approvals, state transitions, tool audits, action outcomes, and artifact references; images and masks remain files with stable references. Streamlit is an API-only client with five views: overview, DQF processing, incident investigation, actions/approvals, and evaluation. The UI does not duplicate quality rules, inference, or workflow transitions.

Quarantine is idempotent; reruns are local simulations; repairs create a separate schema-aligned artifact and preserve the source. Server-side policy checks preconditions and approval records, and state transitions/tool outcomes are audited. The seeded benchmark contains 360 synthetic runs in 120 incident groups. Grouped partitions contain 216 training runs (24 clean fitting runs), 72 validation runs, and 72 test runs (42 positive, 30 negative). Thresholds are selected on validation and frozen before testing. The DIP experiment evaluates three held-out 30×12 fingerprints covering 90 of 108 held-out runs; validity masking leaves 1,024 scored pixels. The matched detection ablation has 43 test runs. AP uses the step-function definition in average_precision_score, not trapezoidal PR-AUC.

TABLE I
INCIDENT DETECTION ON THE FROZEN TEST SPLIT (n = 72, 42 POSITIVE RUNS)

Configuration	TP/FP/FN/TN	Prec.	Rec.	F1
Rule baseline (n = 72)	28/0/14/30	1.000	0.667	0.800
Isolation Forest (structured, n = 72)	41/29/1/1	0.586	0.976	0.732
Matched IF + DQF regions (n = 43)	28/9/0/6	0.757	1.000	0.862

No neural vision network, vector embedding service, production connector, or external scheduler is used. Detection is tree-based; DIP runs through CPU OpenCV; retrieval is local lexical TF–IDF. Saved test outputs include held-out predictions, confusion counts, masks, component metrics, retrieval judgments, RCA cases, and observed timings. Retrieval and RCA medians are measured on the current local environment; no end-to-end production latency or service-level objective is claimed.

V. RESULTS, DASHBOARDS, AND EVALUATION

A. Incident Detection

Table I reports held-out scores on the 72-run test partition (42 injected anomalies and 30 non-anomalous runs). The matched DQF-feature ablation improves thresholded F1 and reduces false positives, while average precision declines. No MTTD is reported because the evaluation uses run-level records and has no streaming alert-time protocol.

B. DIP Segmentation

Table II compares global Otsu (DIP-B0) with adaptive thresholding plus morphological closing (DIP-P1) over the same 1,024 valid cells from three held-out masks. DIP-P1 produces substantially more false-positive pixels and regions. Both recover all 90 positive pixels, but DIP-B0 matches 22/23 ground-truth components at IoU ≥ 0.5 while DIP-P1 matches 1/23. Median CPU processing times are measured per 30×12 fingerprint.

TABLE II
DQF Segmentation on Three Held-Out Fingerprints (1,024 Valid Pixels)

Method	Prec.	Rec.	F1	IoU	ms/image
DIP-B0: Otsu global	0.638	1.000	0.779	0.638	0.325
DIP-P1: adaptive + closing	0.194	1.000	0.324	0.194	0.180

C. Agent, Retrieval, and Operational Metrics

The fixed local retrieval evaluation ranks a relevant runbook section within the top three for 6/7 canonical intent queries (hit@1=hit@3=0.857, MRR=0.857); median measured query time is 0.241 ms and maximum is 0.395 ms. One safe-rerun query is missed, consistent with the lexical retriever's sensitivity to wording. On five evidence-backed synthetic RCA cases, the expected cause is ranked first in 4/5 and appears in the top three in 5/5 (top-1=0.800, top-3=1.000). Median RCA evaluation time is 0.102 ms, with 0.148 ms maximum. Three policy probes (unapproved schema repair, approval by an unauthorized role, and release before verification) were rejected as expected in 3/3 cases; median policy-check time was 0.055 ms (maximum 0.097 ms). These are small fixed-case measurements, not production reliability estimates. Separate integration tests exercise approval gating, source preservation, quarantine, verification, release, restart restoration, and a rerun that remains unresolved when quality defects persist.

The operational judgment sets are deliberately small and handcrafted: seven retrieval queries and five RCA cases. Their measured scores should be read as functional evidence on those cases, not general estimates. No MTTD or end-to-end latency is reported. All benchmark data are synthetic, and the detector test contains only 30 negatives; additional seeds, real labeled incidents, and broader fault distributions are required before making generalization claims.

D. Dashboard and Ablation Interpretation

The implemented Streamlit UI exposes five views: (1) overview of run volume, failures, and incidents; (2) DQF raw, validity, ground-truth, baseline, adaptive masks and overlays; (3) incident evidence, ranked RCA hypotheses and cited runbook sections; (4) action risk, preconditions, approvals, verification and audit timeline; and (5) evaluation results. Each view consumes FastAPI responses, and simulated actions are labeled. The detection ablation improves F1 at the selected threshold but lowers AP; DQF remains corroborative evidence rather than a causal explanation. The presentation includes captures from the running dashboard.

VI. Conclusion and Future Scope

DataGuard AI implements a traceable local workflow from canonical telemetry through quality checks and anomaly detection to visual evidence, RCA, runbook retrieval, policy-gated simulated actions, verification, and persisted audit state. On the reproducible synthetic test set, structured Isolation Forest reached F1=0.732 and AP=0.591; adding DQF-region features on a matched subset raised F1 from 0.789 to 0.862 but lowered AP from 0.764 to 0.673. Global Otsu segmentation outperformed the adaptive/morphological alternative in both pixel and region recovery. Retrieval hit@3 was 6/7 and RCA top-3 accuracy 5/5 on fixed small test sets. Results establish that the modular prototype is implemented and runnable, not that it is production ready.

The evidence is limited to synthetic data and small test sets; it does not establish production-scale reliability or generalisation to real incidents. Saved CSV/JSON outputs include held-out predictions and confusion counts, segmentation and region results, per-query retrieval ranks, RCA cases, and observed local timings. Further work should expand seeds and fault amplitudes, independently label production-like incidents, and measure streaming alert time and end-to-end service latency. Neural segmentation and live connectors are outside this local prototype.

References

[1] S. Schelter *et al.*, "Automating large-scale data quality verification," *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.

[2] J. Ma and H. Wang, "Anomaly detection in sensor data via encoding time series into images," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 36, no. 10, Art. no. 102232, 2024, doi: 10.1016/j.jksuci.2024.102232.

[3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

[4] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Systems, Man, and Cybernetics*, vol. SMC-9, no. 1, pp. 62–66, Jan. 1979, doi: 10.1109/TSMC.1979.4310076.

[5] J. Serra, *Image Analysis and Mathematical Morphology*. London, U.K.: Academic Press, 1982.

[6] Z. Wang and T. Oates, "Encoding time series as images for visual inspection and classification," in *Proc. AAAI Workshops*, 2015.

[7] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[8] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023.